

Listas em Python

Indexação, pertinência, igualdade e fatias

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP — Universidade Federal de Sergipe andreyoshiaki@dcomp.ufs.br

Criar uma lista e acessar seus elementos

Pertinência: o operador `in`

Igualdade: o operador `==`

Fatias

Listas heterogêneas

Colocar a lista plana em ordem

Soluções

**Criar uma lista e acessar seus
elementos**

```
notas = [7.5, 8.0, 6.25, 9.0, 5.5]

print(notas)          # [7.5, 8.0, 6.25, 9.0, 5.5]
print(len(notas))     # 5
print(type(notas))    # <class 'list'>
```

- Os colchetes `[]` delimitam a lista; as vírgulas separam os elementos.
- A ordem em que os elementos foram escritos é preservada.
- `len` devolve a quantidade de elementos.

Referência: Downey (2016), cap. 10, seq. “Uma lista é uma sequência”.

Indexação: cada posição tem dois nomes

0	1	2	3	4
7.5	8.0	6.25	9.0	5.5
-5	-4	-3	-2	-1

```
notas[0]    # 7.5  primeira nota
notas[4]    # 5.5  ultima nota
notas[-1]   # 5.5  ultima nota, contando do fim
notas[-2]   # 9.0  penultima nota
```

Referência: Matthes (2021), cap. 3, seq. “Acessando elementos de uma lista”.

Índice fora da faixa é erro, não aviso



```
notas = [7.5, 8.0, 6.25, 9.0, 5.5]
notas[5]
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

- Uma lista de n elementos aceita índices de 0 a $n-1$.
- Do outro lado, de -1 a $-n$.
- `notas[len(notas)]` é sempre erro; `notas[len(notas) - 1]` é o último.

Referência: Python Software Foundation (2026), Tutorial, seq. 3.1.3.

```
notas = [7.5, 8.0, 6.25, 9.0, 5.5]
```

Escreva o código que imprime:

1. a primeira nota, usando índice positivo;
2. a última nota de duas formas diferentes: uma com `len` e outra com índice negativo;
3. a penúltima nota;
4. a soma da primeira com a última nota.

Antes de executar, responda no papel: o que acontece ao avaliar `notas[5]` ?

Pertinência: o operador `in`

O operador `in` pergunta “está aí dentro?”



```
presentes = ["Ana", "Bruno", "Carla", "Diego"]
```

```
"Carla" in presentes      # True  
"Elisa" in presentes     # False  
"Elisa" not in presentes  # True
```

- O resultado é sempre `True` ou `False` — um valor do tipo `bool`.
- A comparação é feita elemento a elemento, e *strings* distinguem maiúsculas de minúsculas.
- `in` procura o elemento inteiro, não um pedaço dele.

Referência: Matthes (2021), cap. 5, seq. “Verificando se um valor está em uma lista”; PSF (2026), Tutorial, seq. 5.7 “More on Conditions”.

```
presentes = ["Ana", "Bruno", "Carla", "Diego"]
```

Preveja o resultado e só depois confirme no interpretador:

1. "Carla" in presentes
2. "carla" in presentes — explique a diferença para o item anterior;
3. "Ana Souza" in presentes — por que não basta conter o nome?

Agora escreva você:

4. uma expressão que informe se “Elisa” está *ausente*;
5. uma expressão que verifique se o último aluno da lista é “Diego”, **sem usar** in.

Igualdade: o operador ==

== compara conteúdo, posição a posição

```
[1, 2, 3] == [1, 2, 3]    # True
[1, 2, 3] == [3, 2, 1]    # False    a ordem importa
[1, 2, 3] == [1, 2]       # False    o tamanho importa
[1, 2, 3] == [1, 2, 3.0] # True      3 == 3.0
```

- Duas listas são iguais quando têm o mesmo tamanho e elementos iguais nas mesmas posições.
- A comparação de cada par de elementos usa o == do tipo deles.

Referência: Python Software Foundation (2026), Tutorial, seq. 5.8 “Comparing Sequences”.

== é igualdade; is é identidade

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

a == b    # True    mesmo conteúdo
a is b    # False   são dois objetos distintos na memória
a is c    # True    c é outro nome para o mesmo objeto

c[0] = 99
print(a)  # [99, 2, 3] alterar c altera a
```

- `is` pergunta se os dois nomes apontam para o *mesmo* objeto.
- Use `==` para comparar valores; `is` praticamente só para `None`.

Referência: Ramalho (2023), cap. 6, seq. “Identidade, igualdade e apelidos”.

```
gabarito = ["A", "C", "B", "D", "A"]  
resposta = ["A", "C", "B", "A", "A"]
```

1. Qual o valor de `gabarito == resposta`? Por quê?
2. Compare a segunda questão das duas listas com uma única expressão `==`.
3. Escreva um laço que percorra os índices e imprima a **primeira** posição em que as listas divergem.
4. Faça `copia = gabarito`. O que devolvem `copia == gabarito` e `copia is gabarito`?
5. Após `copia[0] = "B"`, imprima `gabarito`. Explique o resultado.

Fatias

```
temperaturas = [21, 23, 26, 28, 30, 29, 25]

temperaturas[0:3]    # [21, 23, 26]    indices 0, 1 e 2
temperaturas[2:5]    # [26, 28, 30]    indices 2, 3 e 4
temperaturas[:3]     # [21, 23, 26]    inicio omitido = 0
temperaturas[4:]     # [30, 29, 25]    fim omitido = ate o final
```

- Em `lista[i:j]`, o índice `i` entra e o índice `j` **fica de fora**.
- Por isso `j - i` é o número de elementos da fatia.
- A lista original não é modificada: a fatia é um objeto novo.

Referência: Matthes (2021), cap. 4, seq. “Trabalhando com parte de uma lista”.

```
temperaturas = [21, 23, 26, 28, 30, 29, 25]

temperaturas[::2]      # [21, 26, 30, 25]    de dois em dois
temperaturas[1::2]     # [23, 28, 29]       começando no índice 1
temperaturas[::-1]     # [25, 29, 30, 28, 26, 23, 21] invertida
temperaturas[:]        # cópia completa
```

- A forma geral é `lista[inicio:fim:passo]`.
- Passo negativo percorre a lista de trás para frente.
- `lista[:]` produz uma cópia — igual pelo `==`, diferente pelo `is`.

Referência: Ramalho (2023), cap. 2, seq. “Fatiamento”.

```
original = [1, 2, 3]
copia = original[:]      # nova lista com o mesmo conteúdo
apelido = original       # outro nome para a mesma lista

apelido[0] = 99
print(original)          # [99, 2, 3]
print(copia)             # [1, 2, 3]
print(copia == original) # False
```

Alterar pelo apelido altera a lista original; alterar a cópia, não.

Referência: Downey (2016), cap. 10, seq. “Apelidos”.

```
temperaturas = [21, 23, 26, 28, 30, 29, 25]
```

1. Obtenha as três primeiras e as três últimas medições, cada uma com uma fatia.
2. Obtenha as medições dos índices 2 a 4, *inclusive*.
3. Obtenha as medições de dias alternados, começando pelo primeiro.
4. Use `==` e uma fatia para decidir se a lista é igual à sua própria inversão.
5. Crie cópia com `[:]` e apelido sem fatia; altere `apelido[0] = 0` e imprima as três listas. Explique o que `==` e `is` respondem para cada par.

Listas heterogêneas

```
aluno = ["Ana Souza", 19, True, [8.5, 7.0, 9.25]]

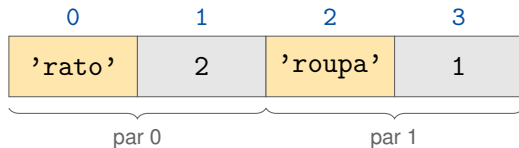
aluno[0]      # 'Ana Souza'    str
aluno[1]      # 19             int
aluno[2]      # True           bool
aluno[3]      # [8.5, 7.0, 9.25] list
aluno[3][1]   # 7.0            segunda nota: indexa a lista de dentro
```

- Cada posição guarda uma referência para um objeto de qualquer tipo.
- Quando o elemento é uma lista, basta indexar duas vezes.

Referência: Ramalho (2023), cap. 2, seq. “Visão geral das sequências embutidas”.

```
aluno = ["Ana Souza", 19, True, [8.5, 7.0, 9.25]]
```

1. Imprima o nome, a idade e a **segunda** nota do aluno.
2. Avalie `19 in aluno` e `9.25 in aluno`. Por que os resultados diferem?
3. Extraia com uma fatia apenas os campos cadastrais (nome, idade e matrícula).
4. Crie outro com exatamente o mesmo conteúdo e compare com `==` e com `is`.
5. Escreva uma expressão que valha `True` quando o aluno estiver matriculado e a média das notas for pelo menos 7,0.



- Sem listas aninhadas: a palavra do par p fica em $2*p$; a contagem, em $2*p + 1$.
- `lista[::2]` devolve só as palavras; `lista[1::2]`, só as contagens.
- `lista.append(x)` acrescenta x ao final — dois `append` inserem um par novo.

Referência: Matthes (2021), cap. 3, seq. “Acrescentando elementos a uma lista”.

```
texto = ["o", "rato", "roeu", "a", "roupa", "do", "rei",  
         "de", "roma", "o", "rato", "roeu", "o", "queijo"]  
contagem = ["o", 3, "rato", 2]  # inicio ja montado, para os itens 1 a 3
```

1. Imprima, por indexação, a palavra e a contagem do par de índice 1.
2. Obtenha com fatias a lista só de palavras e a lista só de contagens.
3. `2 in contagem` vale `True`, mas 2 não é palavra registrada. Escreva a expressão que decide se "rato" é uma **palavra** da contagem.
4. Partindo de `contagem = []`, escreva o laço que percorre `texto` e monta a lista plana: se a palavra já estiver registrada, some 1 à contagem dela; senão, acrescente a palavra e o valor 1.
5. Imprima o relatório palavra: contagem, um por linha.

Colocar a lista plana em ordem

```
lista = [10, 20, 30]
lista[0], lista[2] = lista[2], lista[0]
print(lista)    # [30, 20, 10]
```

- A lista é **mutável**: atribuir a uma posição altera o objeto no lugar, sem criar outra lista.
- A atribuição simultânea troca os dois valores sem precisar de variável auxiliar.
- Na lista plana, trocar um *par* de lugar são **duas** trocas: a palavra e a contagem.
- `max(lista)` e `min(lista)` funcionam sobre uma fatia, porque a fatia é uma lista de verdade.

Referência: Downey (2016), cap. 10, seq. “Listas são mutáveis”.

Exercício 7 — Ordenar por número de ocorrências



```
# contagem de palavras de outro texto, na ordem de primeira aparicao  
contagem = ["sol", 2, "mar", 5, "areia", 1, "vento", 3, "sal", 5, "onda", 4]
```

1. Obtenha a maior contagem sem reordenar nada, usando `max` sobre uma fatia.
2. Escreva as duas trocas que põem o par de índice 2 na frente do par de índice 0.
3. Reordene `contagem` da palavra mais frequente para a menos frequente, movendo sempre o **par inteiro**. A lista deve continuar plana.
4. Confirme com `==` que as contagens ficaram em ordem não-crescente.
5. "mar" e "sal" empatam em 5. Qual dos dois ficou primeiro? Isso era garantido?

Soluções

```
notas = [7.5, 8.0, 6.25, 9.0, 5.5]

print(notas[0])           # 7.5
print(notas[len(notas) - 1]) # 5.5
print(notas[-1])          # 5.5
print(notas[-2])          # 9.0
print(notas[0] + notas[-1]) # 13.0

notas[5]    # IndexError: list index out of range
```

O maior índice válido é `len(notas) - 1`, isto é, 4.

```
presentes = ["Ana", "Bruno", "Carla", "Diego"]

"Carla" in presentes          # True
"carla" in presentes         # False: 'c' e 'C' são caracteres distintos
"Ana Souza" in presentes     # False: in compara o elemento inteiro

"Elisa" not in presentes     # True
presentes[-1] == "Diego"     # True
```

O item 5 mostra que `in` e `==` respondem a perguntas diferentes: pertencer à lista e ser igual a um elemento específico.

Solução do Exercício 3



```
gabarito = ["A", "C", "B", "D", "A"]
resposta = ["A", "C", "B", "A", "A"]

gabarito == resposta          # False: divergem na posicao 3
gabarito[1] == resposta[1]    # True

for i in range(len(gabarito)):
    if gabarito[i] != resposta[i]:
        print("primeira divergencia no indice", i)    # 3
        break

copia = gabarito
copia == gabarito             # True
copia is gabarito             # True: mesmo objeto, nao uma copia
copia[0] = "B"
print(gabarito)               # ['B', 'C', 'B', 'D', 'A']
```

```
temperaturas = [21, 23, 26, 28, 30, 29, 25]

temperaturas[:3]      # [21, 23, 26]
temperaturas[-3:]     # [30, 29, 25]
temperaturas[2:5]     # [26, 28, 30]
temperaturas[::-2]    # [21, 26, 30, 25]

temperaturas == temperaturas[::-1]    # False: nao e palindromo

copia = temperaturas[:]
apelido = temperaturas
apelido[0] = 0
print(temperaturas)    # [0, 23, 26, 28, 30, 29, 25]
print(apelido)         # [0, 23, 26, 28, 30, 29, 25]
print(copia)           # [21, 23, 26, 28, 30, 29, 25]
```

```
apelido == temperaturas    # True    mesmo conteudo
apelido is temperaturas    # True    mesmo objeto
copia == temperaturas      # False   o conteudo mudou em um deles
copia is temperaturas      # False   sempre foram objetos distintos
```

- A fatia `[:]` criou um objeto novo: as alterações feitas por `apelido` não o atingem.
- Logo após a cópia, `copia == temperaturas` ainda era `True`; só a atribuição `apelido[0] = 0` desfez a igualdade.

Solução do Exercício 5



```
aluno = ["Ana Souza", 19, True, [8.5, 7.0, 9.25]]

print(aluno[0], aluno[1], aluno[3][1])    # Ana Souza 19 7.0

19 in aluno          # True: 19 e um elemento de aluno
9.25 in aluno        # False: 9.25 esta dentro de aluno[3], nao de aluno
9.25 in aluno[3]     # True

aluno[:3]            # ['Ana Souza', 19, True]

outro = ["Ana Souza", 19, True, [8.5, 7.0, 9.25]]
outro == aluno       # True
outro is aluno       # False

notas = aluno[3]
aluno[2] and sum(notas) / len(notas) >= 7.0    # True
```

```
contagem = ["o", 3, "rato", 2]

print(contagem[2], contagem[3])    # rato 2    par 1: indices 2*1 e 2*1+1

contagem[::2]    # ['o', 'rato']    so as palavras
contagem[1::2]   # [3, 2]           so as contagens

2 in contagem    # True, mas 2 e uma contagem
"rato" in contagem[::2]    # True e esta e a pergunta certa
2 in contagem[::2]    # False
```

A fatia com passo separa os dois campos e devolve ao `in` a pergunta correta.

```
contagem = []
for palavra in texto:
    achou = False
    for i in range(0, len(contagem), 2):    # visita so os indices pares
        if contagem[i] == palavra:
            contagem[i + 1] = contagem[i + 1] + 1
            achou = True
            break
    if not achou:
        contagem.append(palavra)
        contagem.append(1)

for i in range(0, len(contagem), 2):
    print(contagem[i], ":", contagem[i + 1])
```

```
o : 3      rei : 1
rato : 2   de : 1
roeu : 2   roma : 1
a : 1      queijo : 1
roupa : 1
do : 1
```

- contagem ficou com 20 posições: 10 palavras distintas, 2 posições cada.
- O passo 2 aparece duas vezes com o mesmo papel: em `range(0, len(contagem), 2)` e na fatia `contagem[: :2]`.
- `contagem[: :2]` preserva a ordem da primeira ocorrência de cada palavra.

```
contagem = ["sol", 2, "mar", 5, "areia", 1, "vento", 3, "sal", 5, "onda", 4]

contagem[1::2]          # [2, 5, 1, 3, 5, 4]
max(contagem[1::2])     # 5

trocada = contagem[:]    # copia, para nao perder a ordem original
i, j = 2 * 0, 2 * 2      # inicio do par 0 e inicio do par 2
trocada[i], trocada[j] = trocada[j], trocada[i]
trocada[i+1], trocada[j+1] = trocada[j+1], trocada[i+1]

print(trocada)
# ['areia', 1, 'mar', 5, 'sol', 2, 'vento', 3, 'sal', 5, 'onda', 4]
```

As duas trocas andam juntas: mover só a palavra desalinhará a lista.

Solução do Exercício 7 — item 3



```
ordenada = contagem[:]           # trabalha sobre uma copia
n = len(ordenada) // 2          # quantidade de pares

for p in range(n):
    maior = p
    for q in range(p + 1, n):
        if ordenada[2*q + 1] > ordenada[2*maior + 1]:
            maior = q
    i, j = 2 * p, 2 * maior
    ordenada[i], ordenada[j] = ordenada[j], ordenada[i]
    ordenada[i+1], ordenada[j+1] = ordenada[j+1], ordenada[i+1]

print(ordenada[:2])             # ['mar', 'sal', 'onda', 'vento', 'sol', 'areia']
print(ordenada[1:2])            # [5, 5, 4, 3, 2, 1]
```

```
ordenada[1::2] == [5, 5, 4, 3, 2, 1]    # True  
  
# 'mar' ficou antes de 'sal' -- mas isso nao estava garantido:  
antes  = ["x", 1, "y", 1, "z", 2]  
# depois do mesmo algoritmo:  
depois = ["z", 2, "y", 1, "x", 1]      # 'x' vinha antes de 'y' e agora vem depois
```

- O item 4 usa == entre duas listas: mesmo tamanho, mesmos valores, mesmas posições.
- A troca move o par escolhido para a frente e joga o que estava lá para trás — e é isso que pode inverter dois empatados.
- Quando a ordem de aparição precisa sobreviver ao empate, não basta trocar: é preciso deslocar os pares intermediários.

- **Indexação** — `lista[i]`: um elemento; índices de 0 a $n-1$ ou de -1 a $-n$.
- `in` — pertinência: procura o elemento *inteiro* e devolve `bool`.
- `==` — igualdade de conteúdo, posição a posição; `is` compara identidade.
- **Fatias** — `lista[i:j:k]`: uma *nova* lista; o fim fica de fora.
- **Listas heterogêneas** — cada posição pode guardar um tipo diferente, inclusive outra lista.
- **Lista plana de pares** — campos intercalados; $2*p$ e $2*p+1$ localizam o par p , e o passo 2 separa os campos.
- **Mutabilidade** — atribuir a `lista[i]` altera a lista no lugar; reordenar pares é trocar dois elementos de cada vez.

-  **Allen B. Downey.**
Pense em Python: Pense como um cientista da computação.
Novatec, São Paulo, 2 edition, 2016.
-  **Eric Matthes.**
Curso Intensivo de Python: Uma introdução prática e baseada em projetos à programação.
Novatec, São Paulo, 2 edition, 2021.
-  **Python Software Foundation.**
The python tutorial.
<https://docs.python.org/3/tutorial/>, 2026.
Seções 3.1.3 (Lists) e 5.1 (More on Lists).



Luciano Ramalho.

Python Fluente: Programação clara, concisa e eficaz.

Novatec, São Paulo, 2 edition, 2023.